

Wrocław University of Science and Technology

Faculty of Electronics, Photonics and Microsystems

Field of Study: Electronic and Computer Engineering

BACHELOR THESIS

Title of Thesis:

An AI-Based System for Personalized Career Recommendation

System oparty na sztucznej inteligencji do spersonalizowanego doradztwa zawodowego

Author:

Egemen Goncu

Supervisor:

dr inż. Michał Błędowski

WROCLAW 2026

W pracy przedstawiono projekt, implementację oraz ewaluację systemu opartego na sztucznej inteligencji do spersonalizowanego doradztwa zawodowego. System, wdrożony jako aplikacja webowa na zarządzanej platformie chmurowej, integruje sześć komplementarnych narzędzi (analizę CV, dopasowanie do oferty pracy, sugestie ścieżki kariery, generowanie listu motywacyjnego, przygotowanie do rozmowy kwalifikacyjnej oraz planowanie portfolio), które razem wspierają kandydata w pełnym przebiegu pracy poszukiwania zatrudnienia. Frontend zbudowano w technologiach React 19 i TanStack Start; backend wykorzystuje FastAPI oraz PostgreSQL; wnioskowanie modelu językowego delegowano do usługi Google Vertex AI Gemini 2.5 Flash. W odpowiedzi na obawy dotyczące kosztu, opóźnienia oraz zależności od dostawcy zewnętrznego modelu, rdzeń analityczny systemu obsługuje dwa przełączalne w czasie działania tryby oceny: *tryb mieszany* łączący deterministyczną heurystykę ze strukturalnym wyjściem modelu językowego (40% / 60%), oraz *tryb w pełni heurystyczny* oparty na klasycznych metodach wyszukiwania informacji: ważonej dowodem TF-IDF analizie słów kluczowych z rankingiem BM25, normalizacji umiejętności zgodnej z taksonomią ESCO, dopasowaniu rozmytym opartym na odległości Levenshteina oraz ekstrakcji cech ważonych sekcjami. Kontrolowane studium porównawcze oceniło oba tryby na 100 parach (CV, opis stanowiska) obejmujących sześć ścieżek zawodowych, w czterech rodzinach miar: zgodności wyników (Pearson, Spearman, Kendall), kształcie rozkładu, opóźnieniu oraz koszcie jednego wywołania. Zbiór ewaluacyjny stanowią deterministycznie wygenerowane *syntetyczne CV oraz syntetyczne opisy stanowisk* skonstruowane z rozłącznych słownikowo pul fraz, w których strona CV i strona opisu stanowiska nie współdzieli słownictwa treściowego poza kanonicznymi nazwami umiejętności; ograniczenia konstrukcji zbioru oraz wynikające z nich zagrożenia trafności są jawnie udokumentowane w rozdziale prezentującym wyniki. W obrębie tego syntetycznego zbioru ewaluacyjnego tryb w pełni heurystyczny śledzi tryb mieszany przy Pearsonowskim $r = 0,836$ [95% przedział ufności 0,762, 0,896, bootstrap klastrowy względem identyfikatora CV] oraz Spearmanowskim $\rho = 0,847$ na wyniku ogólnym, działa około dwa do trzech rzędów wielkości szybciej (mediana 25,5 ms wobec 18,2 s) oraz nie ponosi końcowego kosztu po stronie modelu językowego. Korelacja między samym modelem językowym a trybem heurystycznym, odzyskana post-hoc z tożsamości formuły mieszanej w celu odseparowania składowej strukturalnej, wynosi $r = 0,627$ [95% CI 0,476, 0,765]; jest to zgodność krzyżowa po wyłączeniu 0,40-składnika heurystycznego, który tryb mieszany zawiera z definicji. Tryb mieszany zachowuje nieredukowalną przewagę w zakresie dowodów jakości prozy (podwójnik *clarity* $r = 0,395$), co motywuje utrzymanie komponentu językowego obok deterministycznej linii bazowej. Ograniczenia obejmują zakres jednojęzyczny (angielski), syntetyczny zbiór jednego autora oraz pojedynczą konfigurację dostawcy modelu językowego; długoterminowa walidacja podłużna pozostaje poza zakresem. Praca przyszła obejmuje integrację komponentu wyszukiwania wspomagającego, rozszerzenie o CV w języku polskim oraz uzupełnienie heurystyki o pośredni poziom dopasowania semantycznego oparty na transformatorach zdań.

Słowa kluczowe: doradztwo zawodowe, analiza CV, duże modele językowe, wyszukiwanie informacji, taksonomia ESCO, ocena hybrydowa, FastAPI, React.

Abstract (English)

This thesis presents the design, implementation, and evaluation of an AI-based system for personalised career recommendation. The deployed web application integrates six tools: resume analysis, job-description matching, career-path suggestion, cover-letter generation, interview preparation, and portfolio planning. The frontend uses React 19 and TanStack Start; the backend uses FastAPI and PostgreSQL; language-model inference is delegated to Google Vertex AI Gemini 2.5 Flash.

To address cost, latency, and external-provider dependence, the analytical core supports two runtime-switchable scoring modes. The *blended mode* combines a deterministic heuristic with the language model's structured output at 40% / 60% weighting. The *fully heuristic mode* uses classical information-retrieval methods: TF-IDF and BM25 keyword evidence, ESCO-aligned skill normalisation, Levenshtein-style fuzzy matching, and section-weighted feature extraction.

A controlled comparative study evaluated both modes on 100 synthetic resume/job-description pairs spanning six role tracks. The benchmark was deterministically generated under a disjoint-vocabulary construction: resume-side and job-description-side phrase pools share no content-word vocabulary beyond canonical skill terms. Within this benchmark, heuristic mode tracks blended mode at Pearson $r = 0.836$ [95% CI 0.762, 0.896 under cluster bootstrap on resume identity] and Spearman $\rho = 0.847$, while running much faster (median 25.5 ms vs 18.2 s) and incurring zero LLM-side marginal cost. The post-hoc LLM-only-versus-heuristic correlation is $r = 0.627$ [95% CI 0.476, 0.765], after controlling for the 0.40 heuristic share embedded in the blended formula.

The result supports the heuristic mode as a fast, auditable fallback, while the blended mode retains value on prose-quality evidence (clarity sub-score $r = 0.395$). Limitations include English-only evaluation, a single-author synthetic dataset, and one LLM provider. Future work proposes Polish-language ESCO support, retrieval-augmented grounding, and a sentence-transformer intermediate tier.

Keywords: career recommendation, resume analysis, large language models, information retrieval, ESCO taxonomy, hybrid scoring, FastAPI, React.

Acknowledgements

I would like to thank my supervisor, **dr inż. Michał Błędowski**, for the time he invested in shaping the scope of this thesis, the clarity of his feedback during the development of the system, and his suggestion to extend the work with the comparative study reported in Chapter 4. The shape of the engineering and research aspects of this project owes much to those conversations.

I am grateful to the **Faculty of Electronics, Photonics and Microsystems** at Wrocław University of Science and Technology for providing the academic environment in which this thesis was prepared, and to the staff and lecturers whose courses across the *Electronic and Computer Engineering* curriculum gave me the technical foundation on which this work builds.

My thanks to the open-source maintainers of the FastAPI, TanStack, React, and SQLAlchemy projects, whose libraries made it possible for a single developer to ship a system at this scope on a thesis-bound schedule.



Egemen Goncu
Wrocław, May 2026

List of abbreviations and symbols

Abbreviation	Expansion
AI	Artificial Intelligence
API	

Application Programming Interface
APD
Archive of Diploma Theses (USOS-APD)
ATS
Applicant Tracking System
BM25
Best Match 25 (probabilistic relevance ranking function)
CI
Confidence Interval / Continuous Integration, depending on context
CORS
Cross-Origin Resource Sharing
ESCO
European Skills, Competences, Qualifications and Occupations
FastAPI
Python web framework used by the backend
GDPR
General Data Protection Regulation
Gemini
Google large-language-model family used through Vertex AI
HTTP / HTTPS
HyperText Transfer Protocol / HTTP Secure
IDF
Inverse Document Frequency
IR
Information Retrieval
JD
Job Description
JSON
JavaScript Object Notation
JWT
JSON Web Token
KS
Kolmogorov–Smirnov statistical distance
LLM
Large Language Model
NER
Named-Entity Recognition
NLP
Natural Language Processing
OAuth
Open Authorisation / federated-identity protocol
O*NET
Occupational Information Network taxonomy
ORM
Object–Relational Mapping

PDF
 Portable Document Format
 PWr
 Politechnika Wrocławska / Wrocław University of Science and Technology
 RAG
 Retrieval-Augmented Generation
 REST
 Representational State Transfer
 SBERT
 Sentence-BERT
 SQL
 Structured Query Language
 TF-IDF
 Term Frequency–Inverse Document Frequency
 UI / UX
 User Interface / User Experience
 URL
 Uniform Resource Locator
 USOS-APD
 USOS Archive of Diploma Theses
 UUID
 Universally Unique Identifier
 WEFiM
 Faculty of Electronics, Photonics and Microsystems

Symbols (used in formulas in Chapter 4)

Symbol

Meaning

r	Pearson correlation coefficient
ρ, τ	Spearman and Kendall rank-correlation coefficients
k_I, b	BM25 term-frequency saturation and document-length normalisation parameters
τ	Fuzzy-matching similarity threshold
$\text{idf}(t)$	Inverse document frequency of term t
N, M	Number of resumes and number of job descriptions
R, J	A single resume and a single job description
$K(\cdot)$	Keyword extractor function

Contents

Chapter 1 — Introduction

Choosing a career path has become harder as job titles fragment, skill requirements change quickly, and many applications pass through automated screening before a human reviewer sees them [3, 18]. Candidates therefore face two coupled problems: understanding which roles fit their current experience, and presenting that experience in a form that remains legible to both recruiters and algorithmic filters.

Most existing support is fragmented. A candidate may use one tool for resume editing, another for job-board matching, another for a cover letter, and another for interview preparation. Each tool starts from the same resume but rarely carries context into the next step. The thesis addresses this gap by building an integrated AI-based career-support system in which the resume is the shared representation used across all tools.

The immediate motivation was practical rather than abstract. During an internship search, the same resume had to be re-entered into a resume editor, a match-score tool, a cover-letter prompt, and an interview-question generator. The friction was not that any single tool was unusable; it was that each one forgot what the previous step had already established. The system developed for this thesis therefore treats continuity of candidate context as a first-class requirement.

1.1 The career decision problem

Career decision making combines three sub-problems. *Self-assessment* asks the candidate to describe skills, experience, and preferences in a machine-readable form. *Market alignment* asks how that profile compares with job descriptions. *Action planning* asks which changes, projects, or interview stories would improve the candidate’s position.

These problems have usually been studied separately. Career-counselling research treats self-assessment as a structured guidance problem [15]. Information-retrieval research treats market alignment as ranking over job postings [13, 21]. Decision-support research frames action planning as a recommendation task under uncertainty [1, 2]. The practical gap is the absence of a single system in which all three stages share one candidate representation.

The integrated representation matters because one fact can support several decisions. A quantified backend-project bullet may raise the resume quality score, improve fit against a backend job description, justify a stronger cover-letter paragraph, and supply a story for a behavioural interview answer. If the tools are separated, the user must manually preserve and reinterpret that fact; if they share a representation, the system can carry it forward.

1.2 AI-based recommendation systems for careers

Classical recommender systems rely on many users repeatedly interacting with dense item catalogues. Career recommendation is different: job titles are sparse, employer labels are inconsistent, resume evidence is unstructured, and a single user produces few explicit preference signals. Recent career systems therefore treat the task less as collaborative filtering and more as structured matching between a resume and a job description projected into a shared representation [6, 8, 9, 10, 11, 12, 13].

CareerRec [15], for example, applies tree-based classifiers to 2,255 information-technology employee records and reports about 70% accuracy on career-path classification. Broader AI decision-support work [1, 2] shows the same movement from manually engineered features toward learned representations. These systems help select or rank career options, but they usually stop before the candidate has to act on the recommendation. The present thesis extends that workflow by combining assessment, job matching, cover-letter support, interview preparation, and portfolio planning in one application.

This extension is intentionally engineering-oriented. The thesis is not only an offline modelling exercise. It asks whether recommendation logic can be embedded in a deployed workflow where a candidate uploads a resume, receives structured feedback, follows the result into another tool, and later returns to earlier runs through persistent history.

1.3 Resume parsing and job-description analysis

Resume matching starts with structure extraction from free-form text. Early resume parsers used regular expressions and section-header dictionaries to identify names, dates, education, experience, and skills [14]. Such parsers are brittle against unusual formatting but remain valuable as deterministic, inspectable baselines.

Modern parsers rely more heavily on natural-language processing. Named-entity-recognition models extract organisations, dates, and skills; transformer models improve contextual interpretation of ambiguous phrases [4]. Resume2Vec [18] reports a 15.85% improvement in normalised discounted cumulative gain over a non-contextual ATS baseline by using resume embeddings.

Job-description analysis is the demand-side counterpart. Recent work links job postings to standardised taxonomies such as ESCO, the European multilingual classification [8], and O*NET. This normalisation lets “JavaScript”, “JS”, and “ECMAScript” resolve to the same skill. The literature includes top-down ESCO matching and model-based approaches such as JobBERT, ESCOXML-R, and LLM-assisted skill extraction [9, 10, 11, 12, 13]. The system implemented here adopts a top-down strategy: a curated ESCO subset is bundled with the application and used to normalise skills on both sides of the match.

1.4 Heuristic and language-model scoring approaches

The thesis centres on a design tension between two scoring families. The first is *heuristic*: deterministic information-retrieval methods such as TF-IDF [5], BM25 [6], Levenshtein-style fuzzy matching [7], taxonomy lookup [8, 9], and section-weighted feature extraction [18]. These methods are cheap, auditable, and independent of external inference services. Their strongest claim is that they capture much of the first-pass recruiter signal: skill overlap, measurable impact, structure, and completeness.

The second family is *language-model*: LLM services that read a resume and a job description as natural language and return structured reasoning about alignment. Recent recruitment research reports faster screening and richer feedback from LLM agents and semantic feedback loops than from manual or keyword-only review [16, 17, 18, 19]. Their trade-off is per-call cost, higher latency, and a reasoning process that is harder to audit.

This dichotomy motivates the comparative study in Chapter 4. The implemented system exposes both modes through one interface: a *blended* mode that combines heuristic and LLM scores, and a *fully heuristic* mode that skips the LLM. The research question is deliberately bounded: on a controlled, deterministically generated synthetic benchmark of resumes and job descriptions, how much of the blended mode’s discriminative power is recoverable from a strictly classical-IR heuristic, and at what latency and cost ratio? The benchmark limits are stated explicitly in Section 4.1.4.

The implementation order is also disclosed because it affects interpretation. The blended mode existed first; the strong heuristic was designed later after the supervisor proposed a heuristic-only comparison. This creates a post-hoc design-coupling threat that Chapter 4 treats as a validity limitation rather than hiding it.

The value of the comparison is operational. If the heuristic preserves useful ranking behaviour, it can act as a low-cost fallback when the LLM is unavailable and as a live demonstration mode during the diploma defence. If it fails, the system still benefits from the blended mode, but the fallback must be framed only as an emergency response.

The comparison also clarifies the boundary between engineering and research in the project. The engineering task is to deliver a reliable user-facing system. The research task is to characterise one design decision inside that system: whether a transparent scoring baseline remains useful when compared with the LLM-augmented path. The two tasks reinforce each other because the same runtime switch used for the study is also a practical operational control.

1.5 Purpose of the thesis

The objective of the thesis is to develop and evaluate a system that helps users choose suitable career paths based on their skills, experience, and current job-market context, generating personalised suggestions through a combination of classical information-retrieval methods and large-language-model inference.

The scope follows the application approved in the *Archive of Diploma Theses* on 3 February 2026 and has two aspects.

The **engineering aspect** is a deployed web application integrating six tools: Resume Analyzer, Job Match, Career Path, Cover Letter, Interview Q&A, and Portfolio Planner. The implementation uses a Python/FastAPI backend, a React/TanStack Start frontend, PostgreSQL persistence for users and tool-run history, Google Vertex AI Gemini 2.5 Flash for LLM inference, Railway deployment, and operational monitoring through Railway metrics and Sentry.

The **research aspect** is a controlled comparison of two scoring modes. The blended mode combines heuristic and LLM outputs with 40% / 60% weighting. The fully heuristic mode uses TF-IDF/BM25 keyword evidence, ESCO-aligned normalisation, fuzzy matching, section-weighted features, quantification detection, and action-verb scoring. Both modes are evaluated on a synthetic single-author benchmark of 100 resume/job-description pairs using score agreement, distribution shape, latency, and per-call cost.

The thesis does not claim that one mode is universally superior. Its design answer is to keep both modes available: the heuristic mode offers a deterministic fallback when LLM access is unavailable or insufficiently auditable, while the blended mode preserves richer prose and role-fit reasoning.

This dual-mode design also protects the system against two deployment constraints. Budget-limited environments may prefer a local deterministic path for high-volume screening, while audit-sensitive environments may require explanations traceable to fixed scoring rules. The blended path remains the default because it produces richer guidance, but the deterministic path gives the system a defensible degraded mode.

The thesis therefore evaluates the system at two levels. At the product level, success means that six career-support tools operate around one resume representation and persist their results in a coherent workflow. At the analytical level, success means that the score-comparison experiment is reproducible, that the heuristic baseline is specified clearly enough to audit, and that the reported limits prevent the result from being overgeneralised.

1.6 Structure of the thesis

Chapter 2 describes the system architecture: requirements, technology stack, backend layering, frontend routing and state, persistence, security, and Railway deployment.

Chapter 3 describes the six tools and the shared implementation patterns: `run_tool_pipeline`, prompt construction, structured-output validation, caching, retry behaviour, and heuristic fallback.

Chapter 4 presents the comparative study. It defines the synthetic dataset and controls, specifies the strong heuristic v2, reports agreement, distribution, latency, cost, and ablation results, and discusses limitations.

Chapter 5 concludes the thesis by summarising the engineering and empirical contributions, restating the limitations, and identifying future work: Polish-language ESCO support, a sentence-transformer intermediate tier [20, 21], and retrieval-augmented grounding for LLM outputs [25].



The bibliography lists the cited works. Appendix A provides representative source listings, Appendix B documents the synthetic dataset, Appendix C shows production screenshots, and Appendix D records the backend

configuration reference.

Chapter 2 — System architecture

This chapter describes the architecture of the implemented system: requirements, technology stack, backend and frontend structure, persistence, authentication, and deployment. Design rationales are included only where they affect the thesis evaluation or operational behaviour.

2.1 Functional and non-functional requirements

The system was designed to satisfy six functional requirements, one per user-facing tool, and seven non-functional requirements.

2.1.1 Functional requirements

ID / tool	Required behaviour
F1 Resume Analyzer	Assess a resume with overall score, sub-scores, issues, actions, and strengths.
F2 Job Match	Compare a resume with a job description and return score, matched and missing keywords, verdict, and action items.
F3 Career Path	Suggest ranked career paths with rationale, required skills, and progression timeline.
F4 Cover Letter	Generate a tailored cover letter from resume and job description.
F5 Interview Q&A	Generate likely interview questions by category with coaching notes.
F6 Portfolio Planner	Identify skill gaps and concrete portfolio artefact ideas for a target role.

The same uploaded resume must serve all six tools without re-entry. Tool runs must be re-runnable, and previous results must be usable as context for later tools.

2.1.2 Non-functional requirements

The non-functional requirements reflect a single-developer, thesis-scoped deployment.

N1 (Latency). Two targets are separated because the analytical core supports two scoring modes (Section 4.2.9). **N1a:** the heuristic-only path should remain below 50 ms at the 95th percentile. **N1b:** the blended cold path should remain below 25 s at the 95th percentile, dominated by the upstream Gemini call; cached production requests are materially faster as reported in Section 3.9.

N2 (Cost ceiling). LLM cost is bounded by Gemini 2.5 Flash. Fully heuristic mode reduces LLM-side marginal cost to zero.

N3 (Availability). Analytical tools fall back to heuristic-only output after LLM failure and surface a `confidence_note`. Generative tools fail with a structured retry-able error rather than fabricating content.

N4 (Auditability). Each persisted tool invocation stores hashed inputs, full output, timing, access mode, and linkage metadata for later inspection and for the study in Chapter 4.

N5 (Privacy). Logged-in screens load no third-party advertising or analytics scripts. Personal data is encrypted in transit and at rest by the managed provider; passwords are stored as bcrypt hashes.

N6 (Single deployment surface). The thesis deployment runs as one Railway project with frontend, backend, and managed PostgreSQL behind one external origin.

N7 (Demonstrability). The system must be operable from a laptop and must allow the analytical core to switch between blended and fully heuristic modes during a diploma-defence demonstration.

The latency requirement was split after measurement. An earlier single cold-path target below five seconds did not match the observed behaviour of structured LLM calls. Separating N1a and N1b keeps the requirement honest: the heuristic path is evaluated as local computation, while the blended path is evaluated as an upstream-service workflow whose tail latency is outside the application’s direct control.

2.2 Technology stack and rationale

The technology stack is summarised in Table 2.1.

Layer
Choice
Rationale
Backend
FastAPI 0.115, SQLAlchemy 2.0, Alembic
Python-native API, typed validation, mature relational persistence.
Database
PostgreSQL 16 (Railway managed)
Single-node managed database is sufficient for thesis traffic.
LLM provider
Google Vertex AI Gemini 2.5 Flash
Low-cost structured-output model with an asynchronous client.

Frontend

React 19, TanStack Start, Vite 7

SPA workspace, type-safe file routing, fast development build.

Styling and UI

Tailwind 4, Radix UI, Framer Motion

Accessible primitives, utility styling, controlled animation.

Authentication

JWT HttpOnly cookies, Google OAuth

No client-readable tokens and standard refresh rotation.

Operations

Railway, Resend, Sentry

Unified deployment, transactional reset email, lightweight error tracking.

FastAPI was selected over a TypeScript backend because the analytical core benefits from Python’s mature text-processing ecosystem (re, difflib, unicodedata, and optional retrieval libraries) and from the official Google Vertex AI client. Keeping scoring logic in Python reduces the risk of language-boundary differences in the comparative study.

TanStack Start was selected over Next.js because the application is an authenticated workspace rather than a content site. Type-safe file routing and co-located search-parameter state fit the multi-tool workflow, while server components and ISR would add little to the thesis scope.

The deployment stack also reflects the thesis boundary. Railway provides enough operational structure for a live system without requiring a separate platform-engineering layer. PostgreSQL is used instead of a document store because tool history, users, regeneration chains, and workspaces are relational by nature. Sentry and Railway metrics are sufficient for defect investigation at thesis traffic levels.

2.3 Backend architecture

The backend is structured in four horizontal layers (Figure 2.1).

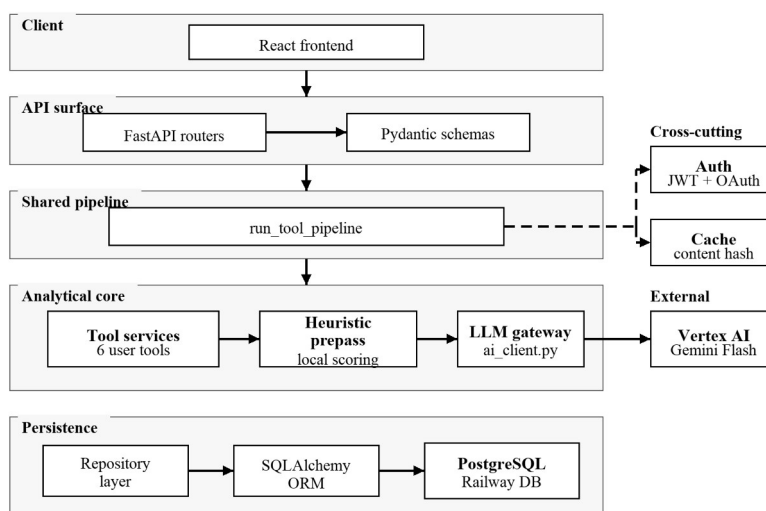


Figure 2.1: Backend layered architecture. The FastAPI surface (routers and schemas) delegates to a service layer that runs every analytical tool through the shared `run_tool_pipeline`, which in turn invokes the heuristic prepass, the LLM gateway (Vertex AI Gemini 2.5 Flash via `ai_client.py`), and the persistence layer (SQLAlchemy 2.0 + Alembic over PostgreSQL).

2.3.1 Routers

Routers live in `app/routers/` and are mounted under `/api/v1`. Each router validates the request body with the relevant Pydantic schema, resolves dependencies such as the current user and database session, invokes a service function, and returns the response. Keeping routers thin makes the tool services and the shared pipeline the main units of behaviour.

2.3.2 The shared tool pipeline

The six tools share `run_tool_pipeline(...)` in `services/tool_pipeline.py`. Its stages are shown in Figure 2.2.

Sanitise. Resume text, job descriptions, and feedback strings are normalised and bounded before reaching prompts.

Cache lookup. A hash of the tool name, sanitised inputs, user identifier, and tool-specific keys is checked against the in-memory cache.

Service call. The tool-specific coroutine performs heuristic extraction, prompt construction, LLM invocation where applicable, and post-processing.

Persistence. Authenticated results are stored as `ToolRun` rows with labels, payloads, linked context, workspace identifiers, regeneration parent ids, and feedback text.

Response assembly. A unified response shape returns the history identifier and access mode.

Observability. Structured start, success, and failure events record tool name, access mode, duration, and error category.

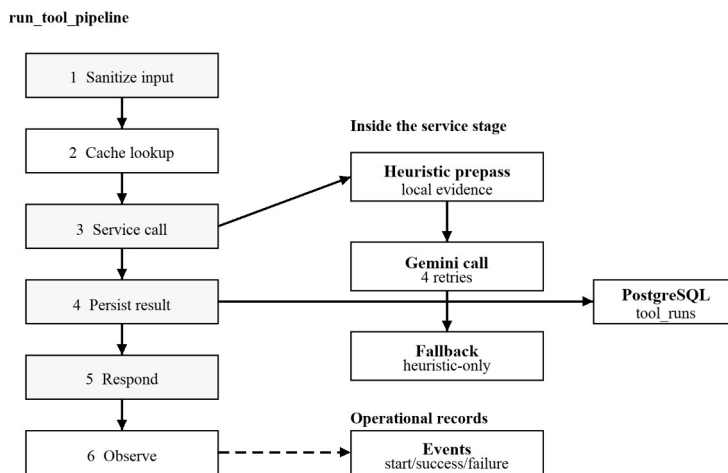


Figure 2.2: Tool pipeline (run_tool_pipeline). Six sequential stages form the cross-cutting wrapper around every analytical tool service: sanitise, cache, service, persist, respond, observe. The LLM gateway is shown as an external dependency exercised inside the service stage, with the four-retry exponential-backoff schedule and the heuristic-only fallback path that activates after the retries are exhausted.

The pipeline gives every tool the same cache, persistence, and observability behaviour. It also lets Chapter 4 reason about mode switching and fallback uniformly rather than per endpoint.

This common wrapper is especially important for regenerated results. A regeneration is just another pipeline run with a parent_run_id, optional feedback, and the same persistence contract. The design avoids hidden per-tool exceptions: whether the run is a resume analysis, a cover letter, or a portfolio plan, the frontend can expect a history identifier and a stable result page.

2.3.3 Tool services and prompt builders

Each tool is implemented through a service module and a prompt module. The service orchestrates the heuristic prepass, invokes complete_structured from services/ai_client.py, and post-processes the validated result. The prompt module assembles the system and user messages from inputs and the locked heuristic payload. This split keeps prompt content versioned but separate from service orchestration.

The service modules are also where mode-specific behaviour is isolated. Analytical services can return a heuristic-only response when SCORING_MODE=heuristic or when the LLM path fails. Generative services do not attempt a heuristic substitute, because a deterministic cover letter or interview answer would be misleading.

2.3.4 The heuristic prepass

Resume Analyzer and Job Match compute a deterministic prepass before the LLM call. The prepass detects sections, bullets, quantified achievements, matched and missing keywords, skill phrases, and five sub-scores: keyword alignment, impact, structure, clarity, and completeness. The payload is appended to the prompt as a locked block that the LLM must treat as ground truth. This stabilises scoring and makes heuristic-only fallback and fully heuristic mode possible without a separate response contract.

2.4 Frontend architecture

The frontend is a single-page application served from the same Railway project as the backend. The backend handles `/api/*`; the frontend handles the remaining routes.

2.4.1 Route structure

Routes live in `frontend/src/routes/` and are generated by TanStack Router. Each tool has an input route such as `/resume` or `/job-match` and a result route at `/<tool>/_/result/$historyId`, where `$historyId` points to the persisted `ToolRun`. Additional routes cover dashboard, account settings, login, password reset, history, and the admin scoring-mode surface.

2.4.2 State management

State is split into three scopes. Server state is held in TanStack Query, including session, history, and loaded results. Workflow state, such as the carried resume and linked result context, is stored in `sessionStorage` through hooks such as `useResumeCarry` and `useSession`. Component state remains local to inputs, sheets, animations, and transient UI flags.

2.4.3 Layout shell and navigation

The application shell uses a dark sidebar, topbar, and light content area. The sidebar groups the six tools into primary, application, and planning clusters. On mobile, navigation collapses into `MobileNav` and `ToolGridSheet`. Tool metadata is centralised in `frontend/src/lib/tools/registry.ts`, so adding a tool requires one registry entry rather than repeated UI edits.

2.5 Data model and persistence

The persistence model contains three primary tables, summarised in Table 2.2.

Table

Purpose

Key columns

users

One row per registered account

id (UUID), email, password_hash (bcrypt) or null for OAuth-only accounts, google_subject for federated identities, timestamps.

tool_runs

One row per tool invocation that the system chose to persist

id, user_id, tool_name, label, result_json, parent_run_id for regeneration chains, linked_context_ids, workspace_id, feedback_text, created_at.

workspaces

Optional grouping of tool runs around a single role search

id, user_id, label, timestamps.

Regeneration never overwrites an existing result: a regenerated run creates a new `tool_runs` row with `parent_run_id` pointing to the earlier row. Guest runs are not persisted to PostgreSQL; they live briefly in memory behind a session cookie, which keeps the database focused on authenticated history.

The `linked_context_ids` field supports the workflow argument of the thesis. When a user invokes a cover letter from a Job Match result, or opens Interview Q&A from an analysed

resume, the new result can record the prior run it depended on. This creates a graph of tool usage without requiring a separate workflow engine.

2.6 Authentication and security

Authentication uses JWTs in HttpOnly cookies: a 30-minute access token and a 7-day refresh token. The refresh token is sent only to `/auth/refresh` and is rotated on every successful refresh to prevent replay. Cookies are marked Secure, SameSite=Lax, and HttpOnly; CSRF tokens are not issued because the API accepts JSON request bodies and the SameSite policy matches the threat model.

Google OAuth is implemented with authlib. Password reset uses a single-use token sent through Resend and bound to the current password hash. Inputs are sanitised before prompt construction (Section 2.3.2), disposable-email domains are blocklisted, and slowapi rate limits login and password-reset endpoints more strictly than tool endpoints.

2.7 Deployment topology

The production deployment runs as a single Railway project with frontend service, backend service, and managed PostgreSQL add-on (Figure 2.3).

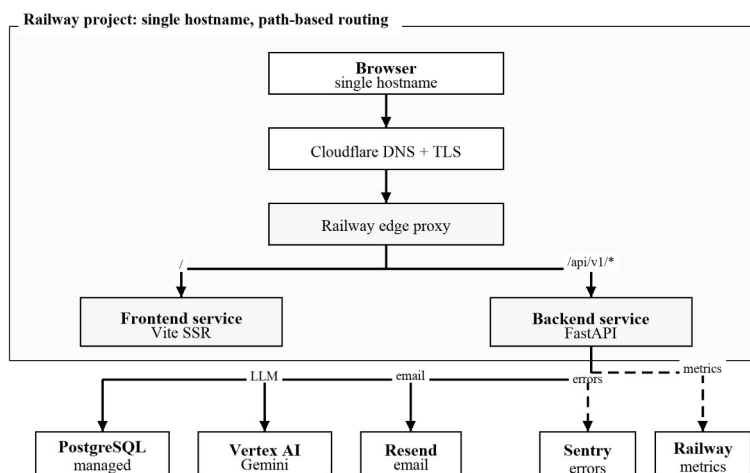


Figure 2.3: Production deployment topology. Browser traffic resolves through Cloudflare DNS and TLS into a single Railway hostname; path-based routing at the Railway edge delivers the `/` path to the Vite-built frontend service and the `/api/v1/*` path to the FastAPI backend service. The backend uses a Railway-managed PostgreSQL add-on for persistence and three external dependencies: Vertex AI Gemini 2.5 Flash for inference, Resend for transactional email, and Sentry for error tracking. Both services emit metrics and breadcrumbs to the observability tier.

The shared hostname removes cross-origin complexity for SPA API calls and simplifies cookie configuration. PostgreSQL backups retain seven daily snapshots. Continuous deployment is tied to the deploy branch, while main remains a development checkpoint. This two-branch model keeps the thesis deployment surface narrow without introducing a CI/CD system larger than the project requires.

Railway was chosen because it keeps frontend, backend, database, logs, and metrics under one operational surface. A split deployment, such as frontend hosting plus a separate backend host, was considered but rejected because it would add CORS, cookie, and routing overhead without improving the thesis evaluation.

The single-origin deployment is also relevant to authentication. HttpOnly cookies are simpler when the SPA and API share one origin, and path-based routing avoids the browser-policy complexity of maintaining separate frontend and backend domains. For a larger product this architecture could be split later, but doing so during the thesis would have added operational risk without strengthening the research result.

The next chapter details the six tools supported by this architecture.

Chapter 3 — Tool implementations

This chapter describes the six tools that compose the user-facing surface of the system: Resume Analyzer, Job Match, Career Path, Cover Letter, Interview Q&A, and Portfolio Planner. They divide naturally into three groups by the role they play in the candidate’s workflow. The *primary* tools (Resume, Job Match) produce numerical assessments and can therefore be run in either the blended or the fully-heuristic mode introduced in Chapter 4. The *application* tools (Cover Letter, Interview Q&A) generate creative artefacts and are LLM-only by construction. The *planning* tools (Career Path, Portfolio Planner) sit between these two extremes; they consume the resume to produce structured forward-looking guidance whose quality depends primarily on the language model’s reasoning over the heuristic prepass.

Before discussing each tool, Section 3.1 isolates the cross-cutting concerns that all six share, so that the per-tool sections can focus on what is specific to each.

3.1 Cross-cutting design

3.1.1 The structured-output LLM client

All LLM calls go through a single coroutine, `complete_structured(system_prompt, user_prompt, schema=None, model_override=None)`, defined in `app/services/ai_client.py`. The function lazily initialises the Vertex AI SDK once per process, dispatches the request to the configured provider (vertex in production), and returns a parsed Python dictionary.

Two reliability mechanisms are built into the client. **Per-call timeout** bounds an individual generation at 120 seconds; this is well above the 95th-percentile latency observed in practice but short enough that a stuck request does not tie up a worker indefinitely. **Exponential backoff with jitter** retries up to four times (at 5, 10, 20, and 40 seconds, plus a small random offset) for transient errors (`TimeoutError`, `RuntimeError`, `json.JSONDecodeError`). JSON-decode errors are retried because Gemini occasionally truncates structured output under load; the retry loop deliberately excludes `ValueError`, which is reserved for unrecoverable configuration errors that do not benefit from retrying. After the final retry exhausts, the exception propagates upward, where each tool service decides how to handle it according to the policy described in Section 3.1.4.

The specific retry schedule (5, 10, 20, 40 seconds) was tuned empirically rather than chosen from a textbook. An earlier version used a tighter 2–4–8–16 schedule; this turned out to retry

faster than Vertex AI’s transient-error window would clear, and the third retry would still hit the same upstream failure. Doubling the base delay to 5 seconds eliminated the pattern almost completely and added at most an extra 30 seconds of waiting in the worst case, which the user-facing UI absorbs comfortably.

3.1.2 The locked-payload prompt pattern

For the analytical tools (Resume Analyzer, Job Match) the prompt builder constructs three blocks: a *system* prompt that establishes role, register, and forbidden behaviours; a *user* prompt that contains the resume and (where applicable) the job description; and a *locked-payload* block that contains the full output of the heuristic prepass. The locked payload includes the deterministic score breakdown, the detected sections, the matched and missing keywords, and the evidence list. The LLM is instructed to treat this block as authoritative ground truth: to not modify the numerical scores, to not contradict the keyword detection, and to respect the evidence the prepass has already surfaced. This pattern stabilises the LLM’s output against prompt perturbations and aligns the LLM’s reasoning with a reproducible baseline. For the comparative study reported in Chapter 4, it is also the reason why disabling the LLM call (the fully-heuristic mode) does not require a parallel codepath; the heuristic prepass already produces a complete response on its own.

3.1.3 Pydantic-validated structured output

Every LLM response is validated against a Pydantic schema defined in `app/schemas/tools.py`. The schema is bidirectional: the same definition that constrains the JSON returned by Gemini also defines the Zod schema used by the frontend (`frontend/src/lib/api/schemas.ts`) to validate the API response before rendering it. Keeping these two definitions in lock-step is enforced through code review rather than through codegen, on the principle that the volume of fields in this project is small enough to not warrant a separate build step. Schema-constrained generation in current language models generally reduces parsing-error rates relative to free-form prompting, with adherence rates that depend on the model and on the prompt style; the operational data observed during the implementation of this thesis is consistent with that direction, although a precise per-cent agreement is not claimed here.

3.1.4 Fallback policy

The system distinguishes two failure modes for an LLM call: a recoverable failure that should produce a degraded but useful response, and an unrecoverable failure that should surface as an explicit error to the user.

For *analytical* tools (Resume Analyzer, Job Match) the recoverable path is taken: when `complete_structured` raises after exhausting retries, the service constructs a complete response from the heuristic prepass alone, using the same payload structure the LLM would have populated but with the deterministic baseline as the source of truth. The user sees the result and a `confidence_note` indicating that the analysis ran in heuristic-only mode. This design satisfies non-functional requirement N3 of Chapter 2.

This fallback path was not added defensively at the design stage; it was added after intermittent transient failures during early testing produced the system’s first user-visible 500-

class error. Once the heuristic fallback was wired up, the same code path made the comparative study described in Chapter 4 trivial to implement: the fully-heuristic mode is, mechanically, the fallback path triggered unconditionally rather than only on failure.

For *generative* tools (Cover Letter, Interview Q&A, Career Path, Portfolio) no fallback is offered. A heuristic-generated cover letter or interview-question list is not a credible substitute for an LLM-generated one; producing such an artefact would be misleading. These tools raise a structured 503 error that the frontend renders as a retry-able failure state.

3.1.5 Caching

The shared pipeline (Chapter 2.3.2) computes a content hash from the tool name, the sanitised inputs, the user identifier, and any tool-specific cache keys. The result of a cache hit is returned without invoking the service function, bypassing the LLM call entirely. The cache is in-memory with a one-hour TTL and is intentionally not shared between processes; this is acceptable for thesis-scoped traffic, and Redis is documented in Chapter 5 as the planned upgrade for V1.1.

3.2 Resume Analyzer

The Resume Analyzer accepts a resume text and an optional target job description, and returns a five-axis quality assessment. It is the most heavily exercised tool in the system; every other tool can be invoked from the result page of the Resume Analyzer with the analysed resume already loaded.

3.2.1 Input contract

The request body contains the raw resume text (free-form, minimum 80 characters), an optional job description text, and an optional feedback string used when the user regenerates with a specific instruction.

3.2.2 Heuristic prepass

`build_resume_prepass(resume_text, job_description)` performs five extractions in a single pass over the resume text.

Section detection. A regular-expression pass identifies common resume section headers (Summary, Experience, Skills, Education, Projects, Certifications) and returns the set of sections present.

Keyword extraction. When a job description is supplied, role-relevant keywords are extracted from the job description and matched against the resume. The result is a pair of lists: matched keywords and missing keywords. Chapter 4 introduces the strong-heuristic v2 of this extraction, which replaces the binary match/miss with a TF-IDF weighted overlap and a fuzzy-matching layer to accept minor surface variation.

Quantification detection. Bullet lines are scanned for numbers, percentages, currency tokens, and time periods that indicate measurable outcomes. The count of quantified bullets is one of the inputs to the *impact* sub-score.

Skill extraction. A curated list of skill phrases is searched against the resume; detected skills are returned as a list. Chapter 4 expands this stage to a top-down lookup against a bundled subset of the ESCO taxonomy [17].

Structural metrics. The total bullet count, the word count, and the number of detected sections are returned as inputs to the *structure*, *clarity*, and *completeness* sub-scores.

The five-axis score breakdown (keyword alignment, impact, structure, clarity, and completeness) is computed from these extractions through `compute_resume_breakdown` in `app/services/quality_signals.py`. Each sub-score is clamped to the integer range [0, 100].

3.2.3 Prompt and post-processing

The resume prompt (`app/prompts/resume.py`) instructs the LLM to act as a senior career coach producing a structured assessment. The locked payload includes the score breakdown and the evidence dictionary. The LLM is asked to populate four narrative fields (a one-line headline, a verdict, a confidence note, and a list of strengths), to expand the deterministic issue list into prose with `why_it_matters`, `evidence`, and `fix` fields, and to produce up to three prioritised top actions that may be different in framing from the issues. When the user has specified a target role label in the job description, the LLM additionally produces a `role_fit` block with a fit score and a rationale.

The service then computes the blended score (heuristic 40%, LLM 60%) following the formula in Section 4.2 and returns a unified response object. When the heuristic and LLM overall scores diverge by more than 20 points, an explicit `confidence_note` is added to the response surface so that the user understands that two independent estimates disagreed.

3.3 Job Match

The Job Match tool accepts a resume and a job description (provided as text or as a URL) and produces a structured match assessment.

3.3.1 Input contract

The job description input may be a URL pointing to a public job posting. When a URL is supplied, the backend attempts to scrape the description through a layered strategy: a primary BeautifulSoup pass for static markup, a Playwright fallback for JavaScript-heavy posting boards, and a graceful paste prompt presented to the user when both attempts fail. The scraped text is then treated identically to a pasted text input.

3.3.2 Heuristic prepass

The Job Match prepass extends the Resume Analyzer prepass with a single additional output: a deterministic match score computed from the matched-keyword and missing-keyword lists through `compute_match_score` in `app/services/quality_signals.py`. The function returns a value in [0, 100] derived from the matched-to-total ratio. The deterministic verdict (strong, borderline, stretch) is read off this score by `job_match_verdict`.

3.3.3 Prompt and post-processing

The Job Match prompt (`app/prompts/job_match.py`) instructs the LLM to produce a structured requirement-by-requirement breakdown. For each detected role requirement, the LLM populates an importance (must / preferred), a status (matched / partial / missing), a one-sentence resume-evidence statement, and a suggested fix. The locked payload supplies the matched and missing keyword lists; the LLM is asked to expand them into role-realistic

phrasing rather than echoing back the raw tokens. The response also includes a recruiter-style summary and a list of tailoring actions ranked by expected impact.

Because the deterministic match score is locked, the LLM cannot move it; this preserves comparability across regenerations and is also the property that makes the fully-heuristic mode of Chapter 4 a clean substitution rather than a re-implementation.

3.4 Career Path

The Career Path tool produces forward-looking career suggestions from the resume alone. Unlike Resume Analyzer and Job Match, it does not consume a job description; the user instead optionally supplies a free-form preference string (“interested in management”, “want to stay technical”) that is folded into the prompt.

3.4.1 Heuristic prepass

The prepass extracts three coarse profile signals from the resume: an inferred *discipline* (backend-engineering, frontend-engineering, full-stack-engineering, data-analytics, product-design, etc., among ~10 supported categories), an inferred *seniority* (junior, mid, senior), and an estimated *years of experience*. Each inference is conservative; when confidence is low, the prepass returns the most general label.

For every discipline, a curated DISCIPLINE_TARGET_SKILLS mapping defined in `app/services/career_recommender.py` lists six aspirational skills typical of that discipline at the next seniority level. This mapping plays the role of a deterministic skills floor for the LLM’s reasoning, similar to the locked payload in the analytical tools but lighter.

3.4.2 Prompt and post-processing

The career prompt instructs the LLM to produce three to five candidate paths, each with a path label, a one-paragraph rationale grounded in the resume, a list of aligned strengths, a list of skills to develop (with the discipline-target list available as a hint), and an indicative timeline expressed in seniority steps rather than calendar months. The output is intentionally not numerical; the comparative study in Chapter 4 does not include the planning tools.

3.5 Cover Letter

The Cover Letter tool produces a tailored cover letter from a resume and a job description. There is no heuristic prepass that would produce a letter, so this tool is LLM-only.

3.5.1 Prompt and post-processing

The prompt asks the LLM to write a letter in three explicit structural sections: an opening that names the role and the candidate’s central value proposition, a middle paragraph that grounds two or three claims in resume evidence, and a closing that proposes a concrete next step. The LLM is instructed to write in a register that the user can adapt (neither overly formal nor casual) and to avoid clichés that recruiters report fatigue with (the prompt includes an explicit blocklist of phrases such as “team player”, “synergy”, and “passionate self-starter”). The response also includes three short alternative opening lines, allowing the user to swap the opening without regenerating the whole letter.

3.6 Interview Q&A

The Interview Q&A tool produces a curated list of likely interview questions for a given resume and job description. It is LLM-only.

3.6.1 Prompt and post-processing

The prompt asks the LLM to cluster the generated questions into three categories (*behavioural*, *technical*, and *role-specific*) with a target of five to seven questions per category. Each question carries a one-sentence coaching note that names the underlying competency the interviewer is probing for and suggests the kind of evidence the candidate should be ready to recall.

A practice mode is available as a follow-up flow: the user picks a question from the generated list and supplies a written practice answer. The system then sends the question, the candidate's answer, and the resume to a cheaper Gemini model (configurable through `LLM_PRACTICE_MODEL`) and returns structured feedback. This is the only place in the system where the LLM model is overridden; the override exists because practice feedback is invoked frequently during a single user session and the cost of using the same model as the rest of the system would be disproportionate to the value.

3.7 Portfolio Planner

The Portfolio Planner tool produces a prioritised list of artefact ideas and skill gaps for a given resume and target role. The output is forward-looking and structural rather than evaluative.

3.7.1 Prompt and post-processing

The prepass detects the resume's discipline and seniority (the same inference used by Career Path) and identifies up to ten skills that the resume *does not* currently support but that the target role requires. The prompt then asks the LLM to produce three to five concrete artefact ideas, each with a working title, a brief description, the skills it would demonstrate, an estimated effort tier (small / medium / large), and a suggested presentation format (GitHub repository, case study, demo video, written post). The response also returns a sequenced plan that orders the artefacts by expected return on effort.

3.8 Persistence, regeneration, and result history

Every tool result that runs in *authenticated* mode is persisted as a ToolRun row through the shared pipeline (Chapter 2.3.2). The persistence layer adds three behaviours that are visible to the user.

Result pages by history identifier. The result of a tool run is rendered at `/<tool>_result/<historyId>`, where the history identifier is the primary key of the ToolRun row. The user can return to a result page later by visiting the URL directly, share the URL with a collaborator who has access to the same account, or reach the result through the *History* page that lists every run in reverse-chronological order.

Regeneration as a parent-child chain. When the user regenerates a result with feedback, a new ToolRun row is written whose `parent_run_id` points to the original. The history page renders this as a chain rather than as two unrelated entries, making it possible to walk back through earlier iterations of a single piece of analytical work.

Cross-tool linkage. A tool may consume the result of a previous tool as part of its input. For example, the Cover Letter can be invoked with the Job Match result already in hand. The pipeline records the linked run identifiers on the new tool run, producing a graph of dependencies that supports the workflow-level reasoning that the user research informed.

3.9 Reliability behaviour observed in production

In production traffic during the development period, the analytical tools (Resume Analyzer, Job Match) exhibited a heuristic-only fallback rate below one percent of all calls; the LLM-call retry policy (Section 3.1.1) absorbs nearly all transient failures. The generative tools (Cover Letter, Interview Q&A, Career Path, Portfolio) showed a slightly higher hard-error rate because they cannot fall back; the user-visible failure mode is a structured retry prompt rather than a fabricated artefact.

Among the six tools, the one that took the longest to stabilise was the Interview Q&A practice mode. The first version invoked the same Gemini model used by the rest of the system; the per-session cost ran high enough during heavy practice usage that the cheaper-model fallback path described above was introduced (the `LLM_PRACTICE_MODEL` configuration is the result of that work). The Cover Letter tool produced the opposite kind of surprise; the structural prompt with the explicit blacklist of clichés produced usable letters on the first end-to-end iteration and required only minor adjustments after a small number of dogfooding runs.

The blended scoring path (heuristic 40%, LLM 60%) and the heuristic-only fallback are exercised by the same code in the analytical services. The fully-heuristic mode introduced in Chapter 4 reuses the same fallback path and adds an administrative toggle that bypasses the LLM call unconditionally. This is the smallest possible change that satisfies non-functional requirement N7 from Chapter 2: runtime switchability between the two scoring modes for the comparative study reported next.

Chapter 4 — Conducted studies and results

This chapter reports the empirical comparison between the two analytical scoring modes implemented in the system: *blended mode* (40% heuristic, 60% language-model output) and *fully heuristic mode* (classical information retrieval only). The research question, introduced in Section 1.4, is how much of the blended mode’s discriminative power is recovered by a strictly non-neural heuristic on a controlled synthetic evaluation set, and at what latency and cost ratio.

4.1 Methodology

4.1.1 Evaluation dataset

The evaluation dataset contains $N = 30$ resumes and $M = 20$ job descriptions across six role tracks: backend engineering, frontend engineering, full-stack engineering, data analytics, product design, and product management. Each track contains five resumes at junior, mid, and senior levels and three or four job descriptions.

The dataset is generated deterministically by `scripts/synthesise_eval_dataset.py` with `SEED = 42`. Resume content is drawn from track-specific `resume_phrases` and job-description content from separate `jd_phrases`; after stop-words and role-domain skill terms are removed, the two pools are lexically disjoint. This Stage G construction mitigates shared-vocabulary leakage (T1)

while preserving legitimate skill overlap through canonical labels. Synthetic generation avoids privacy and licensing issues but limits external validity, as discussed in Section 4.4.

The harness pairs each resume with every job description in the same role track. Four tracks contain three job descriptions and two tracks contain four, giving $5 \times 3 \times 4 + 5 \times 4 \times 2 = 100$ pairs. Each pair is run once in blended mode and once in fully heuristic mode, producing 200 response objects for the metrics in Section 4.3. Appendix B documents the dataset structure.

4.1.2 Metrics

Four metric families are computed per pair.

Score agreement. Pearson r measures linear association between the two modes’ overall scores. Spearman ρ and Kendall τ measure rank agreement, which is more relevant when the score is used as a screening signal. The value $r \approx 0.7$ from retrieval-quality discussion [6] is used only as a reference anchor, not as a formal hypothesis.

Score distribution. The Kolmogorov–Smirnov distance compares the overall score distributions and shows whether the heuristic shifts or compresses the score scale even when ranking is similar.

Latency. Wall-clock service time is measured from request entry into the tool service until a complete response is returned. Network round-trip time between browser and backend is excluded.

Per-call cost. Marginal LLM-side cost is estimated in US dollars. Blended mode incurs Gemini 2.5 Flash token cost; fully heuristic mode incurs no third-party inference charge.

4.1.3 Controls

Three controls isolate the comparison. The same content hash is used for both modes per pair, and `RESULT_CACHE_ENABLED=false` prevents cache reuse from contaminating latency. The same heuristic implementation, selected by `HEURISTIC_VERSION`, is used by both modes, so the only experimental difference is whether `complete_structured` is invoked. The LLM temperature is fixed at 0.0, removing sampling variance from the blended-mode run.

4.1.4 Threats to validity

Four threats are tracked so that the interpretation of Section 4.3 remains bounded.

T1 (Shared-vocabulary leakage between resume and job-description pools, MITIGATED). The early synthesis script drew resume bullets and job-description responsibilities from the same per-track list. In pair `r-backend-01__jd-backend-02`, the sentence “designed and shipped REST APIs serving production traffic” appeared on both sides, and the leakage-version keyword-axis correlation was Pearson $r = 0.941$. The regenerated dataset separates `resume_phrases` and `jd_phrases`; `scripts/verify_disjoint_pools.py` reports mean per-pair content-token overlap falling from 11.82 to 0.44, a $27\times$ reduction, with zero pairs above the >3 -token threshold instead of 97. The results below use the post-mitigation dataset; leakage artefacts remain archived as `thesis/eval-dataset-with-leakage.json` and `thesis/eval-results-with-leakage.json`.

T2 (Self-correlation between compared modes, MEASURED). Since blended score = $0.40 \times \text{heuristic} + 0.60 \times \text{LLM}$, the headline blended-versus-heuristic correlation includes a

structural floor. Because the heuristic payload is locked inside the blended prompt, the LLM-only score is recoverable without new API calls: $llm_only = (blended - 0.4 \times heuristic) / 0.6$. Section 4.3.1’ reports this baseline.

T3 (Non-independent paired observations, ADDRESSED). The 100 observations are not independent because each resume appears in three or four pairs and each job description in five pairs. Pearson confidence intervals are therefore computed by cluster bootstrap on `resume_id` with 2 000 resamples and `seed=42`.

T4 (Post-hoc heuristic design, DISCLOSED). The strong heuristic v2 was designed after the blended pipeline existed. The agreement in Section 4.3 therefore partly reflects design coupling. A fully closed comparison would require designing the heuristic against a held-out reference; this is outside the Bachelor-thesis scope.

T1, T2, and T3 are mitigated by regeneration, LLM-only decomposition, and clustered intervals. T4 remains a disclosed limitation.

These threats also define how the results should not be read. The chapter does not claim population-level hiring validity, fairness, or universal superiority of one scoring family. It reports a controlled within-set characterisation of the implemented system, using a reproducible benchmark whose construction is transparent enough to audit.

4.2 The strong heuristic (v2)

The lightweight prepass from Chapter 3 is sufficient for fallback behaviour but too weak as a research baseline. The strong heuristic v2 replaces binary keyword matching and unweighted averaging with classical information-retrieval components while remaining strictly non-neural.

4.2.1 TF–IDF–weighted keyword evidence

The v2 heuristic weights keyword evidence by inverse document frequency, so rare technical terms contribute more than common words [5, 6]. Keyword alignment is computed as

$$[(R, J) ;=, ;, 100,]$$

where R is the resume, J is the job description, and $K(\cdot)$ extracts the keyword set. BM25 is implemented in `app/services/quality_signals_v2.py` as `_bm25_score` with $k_1 = 1.5$ and $b = 0.75$. Production uses a bundled ESCO-derived baseline IDF; the evaluation harness recomputes IDF from the job descriptions in the benchmark.

4.2.2 ESCO-aligned skill normalisation

The heuristic normalises surface variants such as *Postgres*, *PostgreSQL*, and *postgres database* to one canonical ESCO label [8, 9, 12]. A curated ESCO subset of roughly one hundred digital and creative-industry entries is bundled as JSON and structured for expansion to broader production coverage. Resume and job-description text are scanned for surface variants, mapped to canonical labels, and intersected.



This top-down design was chosen over a learned skill extractor because it keeps the fully heuristic path inspectable. Every accepted match can be traced to a surface variant in the bundled taxonomy or to the fuzzy rule described below. That property is useful when heuristic mode is presented as an auditable alternative to the LLM path.

4.2.3 Fuzzy matching

For skills or keywords outside the bundled ESCO subset, v2 applies a Levenshtein-style fuzzy pass [7] using Python’s `difflib.SequenceMatcher`. A job-description keyword is counted when it appears exactly, maps through ESCO, or reaches similarity threshold $\tau = 0.85$ against a resume token. The threshold admits typographic variants such as *Javacript* for *JavaScript* while rejecting unrelated tokens.

4.2.4 Section-weighted features

Skill evidence is weighted by resume section because a skill in *Skills* carries a different signal from the same word in *Education* or a project description [18]. Matches in *Skills* contribute 1.0, *Experience* or *Projects* 0.7, *Summary* 0.5, and *Education* 0.3. The weighted match total is normalised against the maximum possible weighted match.

4.2.5 Quantification regex

The impact sub-score detects measurable outcomes through one composite regular expression covering bare integers, percentages, currency amounts, time periods, magnitudes such as k/M/B/×, user or customer counts, and team-size descriptors. The count of quantified bullets enters the score with logarithmic saturation.

4.2.6 Action-verb scoring

The clarity sub-score counts the fraction of bullets whose leading token belongs to a curated action-verb dictionary of approximately 190 entries compiled from university career-services resources. Action verbs such as “led”, “shipped”, and “automated” are treated as stronger ownership evidence than passive or filler openings.

4.2.7 Score combination

The five sub-scores (keyword alignment, impact, structure, clarity, completeness) are combined as a weighted mean: 0.30, 0.25, 0.15, 0.15, and 0.15. These weights are author-selected rather than survey-derived. Section 4.3.5 reports an equal-weight sensitivity check.

4.2.8 Implementation notes

The strong heuristic v2 lives in `app/services/quality_signals_v2.py` alongside the lightweight `quality_signals.py`. `HEURISTIC_VERSION=v2` selects the strong implementation for the analytical tools and is the default for the experimental run reported here.

4.2.9 The administrative toggle

Runtime switchability is provided by SCORING_MODE, which accepts blended and heuristic. An in-memory override in app/services/runtime_settings.py is updated through POST /api/v1/admin/scoring-mode and read by analytical services on every request. The override supports live demonstration during the diploma defence but resets on process restart, leaving the environment variable as the configuration of record.

4.3 Results

The evaluation harness scripts/eval_scoring.py produced 100 paired observations. Both modes succeeded on every pair, and scripts/analyze_eval_results.py generated the tables below from thesis/eval-results.json.

4.3.1 Score agreement

Table 4.1 compares post-mitigation and leakage-version agreement.

Statistic	
Post-mitigation	
Pre-mitigation (leakage)	
Pearson r	
0.836 [95% CI 0.762, 0.896]	
0.727 [95% CI 0.609, 0.814]	
Spearman ρ	
0.847	
0.708	
Kendall τ	
0.669	
0.526	

The post-mitigation Pearson r is higher than the leakage-version value

	even though leakage was expected to inflate keyword overlap. The per- sub-score table explains the result: lexical leakage added noise to some non- keyword axes, while the disjoint constructi on preserved legitimate canonical skill overlap.
--	---

Sub-score	
Post-mitigation Pearson r	
Pre-mitigation Pearson r	
Spearman ρ	
Kendall τ	
keywords	
0.937	
0.941	
0.946	
0.845	
impact	

0.430
0.561
0.447
0.332
structure
0.830
0.440
0.865
0.804
clarity
0.395
0.475
0.373
0.289
completeness
0.756
0.691
0.793
0.689

The agreement is strongest on keywords and structure and weakest on clarity, the axis where the language model sees prose-quality evidence that the heuristic cannot inspect. A correlation-space sanity check is consistent with the blended formula: using $r(\text{LLM-only}, \text{heuristic}) = 0.627$, the equal-variance prediction is $r \approx 0.857$, close to the observed $r(\text{blended}, \text{heuristic}) = 0.836$.

The keyword axis being stable across leakage and disjoint versions is important. It shows that the post-mitigation result is not simply an artefact of repeated responsibility sentences; both modes still respond to legitimate skill labels such as Python, React, BM25, and PostgreSQL. By contrast, the clarity axis remains low because a deterministic rule set cannot assess ownership and role-fit prose with the same nuance as the language model.

4.3.1’ LLM-only baseline (T2 mitigation)

The locked-payload design makes the LLM-only score recoverable as:

--

$$\text{LLM-only} = (\text{blended} - 0.4 \times \text{heuristic}) / 0.6$$

Pair
Pearson *r*
95% CI (cluster bootstrap)
Spearman ρ
blended vs heuristic

0.836
[0.762, 0.896]
0.847
LLM-only vs heuristic
0.627
[0.476, 0.765]
0.596
blended vs LLM-only
0.952
[0.923, 0.973]
n/a

The recovered LLM-only score has mean 71.29, standard deviation 8.45, median 73.33, 5th percentile 54.62, and 95th percentile 82.00. The 0.627 correlation is the honest cross-mode agreement after removing the 0.40 heuristic share embedded in the blended score; the 0.836 headline remains relevant for the operational question of what the runtime toggle changes.



The two numbers answer different questions. The LLM-only value asks how far classical IR agrees with the language model when the shared heuristic component is removed. The blended-versus-heuristic value asks what an operator loses when switching the deployed system from default blended mode to fully heuristic runtime mode.

4.3.2 Score distribution

Table 4.2 reports the distribution of overall scores across all 100 pairs.

Statistic
Blended mode
Heuristic v2 mode

Mean
71.84
72.66
Std. dev.
7.19
7.08
Median
73.00
74.50
5th percentile
58.00
59.00
95th percentile
81.00
82.00
KS distance
0.100
n/a

The means differ by 0.82 points and the medians by 1.50 points, so the aggregate score distributions are close. The KS distance of 0.100 is half the leakage-version KS distance of 0.200. Figure 4.1 visualises the two distributions.

Figure 4.1 — Distribution of overall scores across 100 (resume, JD) pairs

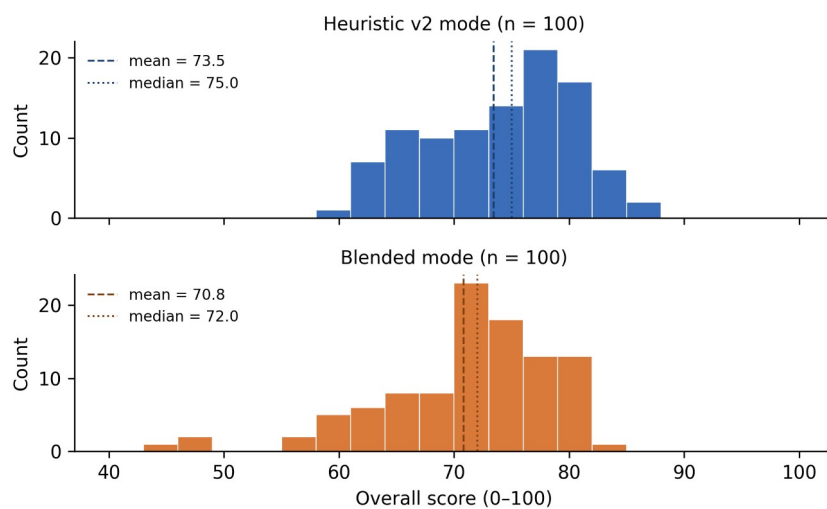
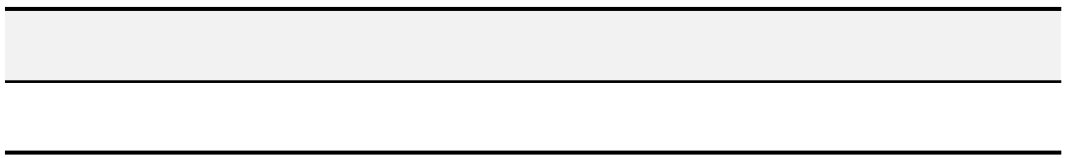


Figure 4.1: Distribution of overall scores across 100 (resume, JD) pairs, by scoring mode.





*Figure 4.1:
Distribution of
overall scores
across 100
synthetic (resume,
JD) pairs. Top:
heuristic-only
mode (mean
73.45, median
75.0). Bottom:
blended mode
(mean 70.84,
median 72.0). The
dashed vertical
line marks the
mean and the
dotted vertical line
marks the median
in each panel.*

4.3.3 Latency

Table 4.3 reports service-side latency for one tool invocation.

Latency

Blended mode

Heuristic v2 mode

Ratio

Median

18 232.1 ms

25.5 ms

714×

95th percentile

24 713.7 ms

32.1 ms

770×

Maximum observed

65 926.6 ms

34.1 ms

1 933×

--	--

Blended latency is dominated by Gemini structured-output inference over approximately 608 input tokens plus prompt overhead and 1 118 output tokens. The maximum of 65 926.6 ms reflects one retry-after-timeout case handled by the retry policy. Heuristic v2 runs in tens of milliseconds and satisfies the N1a target from Section 2.1.2.

4.3.4 Per-call cost

Cost uses the Vertex AI Gemini 2.5 Flash rates in effect at measurement time: input \$0.075 / 1 M tokens and output \$0.30 / 1 M tokens.

Cost item

Blended mode

Heuristic v2 mode

Input tokens (avg., est.)

608

0

Output tokens (avg., est.)

1 118

0

Per-call USD (avg.)

\$0.00038

\$0.00000

Per-call USD (95th pct.)

\$0.00042

\$0.00000

1 000 calls / day projected monthly bill

\$11.43
\$0.00

The estimator follows the ~4 characters per token rule and does not include all server-side prompt overhead, so the claim is order-of-magnitude rather than billing-grade. At 1 000 analyses per day, blended mode projects to roughly \$11.43 per month in LLM-side spend on this dataset; heuristic mode is free on the LLM side.

--

This absolute cost is small at thesis scale, but the ratio matters for mode design. A demonstration, outage fallback, or high-volume preview flow can use the heuristic path without accumulating third-party inference cost. The blended path can then be reserved for cases where richer explanations are worth the additional latency.

4.3.5 Sensitivity to the score-combination weights

The author-selected weights from Section 4.2.7 are compared with equal weights recomputed offline from the same per-sub-score breakdown.

Weighting

Pearson *r*

Spearman ρ

Author-selected (0.30/0.25/0.15/0.15/0.15)

0.836

0.847

Equal (0.20 across all five)

0.813

0.823

--

Pearson
moves by Δ
= -0.023
and
Spearman
by Δ =
 -0.024 .
Both
remain
above the
0.7
reference
threshold,
so the result
is not
dependent
on the exact
weighting
choice.

4.3.6 Heuristic-enhancement ablation (Stage H)

Stage H tested whether targeted heuristic enhancements could close the LLM-only residual without adding a deployed neural dependency. A separate `quality_signals_v3.py` module added five gated signals: SBERT semantic fallback with all-MiniLM-L6-v2 at cosine threshold 0.65 (H.1), STAR-format detection (H.2), cross-axis coherence checks (H.3), bigram/trigram matching (H.4), and ESCO expansion from 107 to 354 entries (H.5). Each variant was compared against the same post-hoc LLM-only baseline, so no additional LLM calls were issued.

Variant

Pearson r vs LLM-only

95% CI

Spearman ρ

Δ vs v2

heuristic-v2 (BM25 + ESCO + fuzzy)

0.627

[0.470, 0.758]
 0.596
 baseline
 heuristic-v3 with no Stage H features
 0.627
 [0.470, 0.758]
 0.596
 +0.000
 + bigram / trigram (H.4)
 0.629
 [0.470, 0.758]
 0.583
 +0.002
 + SBERT semantic fallback (H.1)
 0.623
 [0.452, 0.759]
 0.573
 −0.004
 + STAR detection (H.2)
 0.621
 [0.463, 0.755]
 0.596
 −0.006
 + ESCO expansion 107 → 354 (H.5)
 0.617
 [0.461, 0.752]
 0.578
 −0.009
 + cross-axis coherence (H.3)
 0.605
 [0.438, 0.746]
 0.568
 −0.022
 heuristic-v3 (full, all five)
 0.590
 [0.415, 0.735]
 0.556
 −0.036

The result is a controlled null with small negative drift in the full configuration. v3-empty reproduces v2 exactly to three decimals, validating the module boundary. Single-feature deltas range from +0.002 to −0.022 and remain inside the cluster-bootstrap interval width. The full v3 stack falls to $r = 0.590$ ($\Delta = -0.036$), suggesting that small false-positive effects compound.

The likely explanation is that the synthetic benchmark already exposes canonical skill overlap, so extra keyword recall through SBERT, n-grams, or ESCO expansion adds little. The

remaining gap to the LLM-only score reflects prose quality, narrative coherence, and role-fit judgement rather than missing keyword evidence. The ablation is reproducible through `scripts/eval_scoring_v3_ablation.py`, `scripts/analyze_v3_ablation.py`, `backend/app/services/quality_signals_v3.py`, and `backend/app/data/esco_skills_expanded.json`; PyTorch and sentence-transformers stay in `backend/requirements-thesis-eval.txt`, outside the production requirements.

This null result is still useful. It prevents the future-work section from overstating sentence-transformer matching as an obvious next production step. On this benchmark, the strict v2 heuristic is already near the available keyword-and-structure ceiling, and the unresolved gap sits in evidence classes that require either different features or a different evaluation surface.

4.3.7 Implementation note on Stage H reproducibility

The ablation reuses `thesis/eval-results.json` from Stage G and recovers LLM-only scores through the identity in Section 4.3.1'. It runs in approximately 4 minutes on commodity hardware: about 110 s for SBERT first load and inference, and about 10 s combined for deterministic variants. This keeps Stage H as a research artefact rather than a production dependency.

4.4 Discussion and limitations

Within this synthetic evaluation set, the fully heuristic mode recovers most of the blended mode's aggregate discrimination: Pearson $r = 0.836$ [95% CI 0.762, 0.896], Spearman $\rho = 0.847$, median latency 25.5 ms rather than 18.2 s, and zero LLM-side marginal cost. After removing the structural 0.40 heuristic share in the blended formula, LLM-only versus heuristic agreement is $r = 0.627$ [95% CI 0.476, 0.765]. The deterministic baseline is therefore strong enough to justify deployment as a fallback and runtime mode, while the blended mode remains valuable for qualitative explanations and clarity-sensitive judgement.

Four limitations constrain the claim. **Single-language scope:** all evaluation resumes and job descriptions are in English, although ESCO is multilingual. **Single-subject evaluation:** one author synthesised all 30 resumes, so stylistic regularities may not generalise. **Single LLM provider:** the blended mode uses Gemini 2.5 Flash only. **Bias and fairness:** the benchmark has no demographic annotations and cannot support disparate-impact analysis; production deployment beyond thesis scope would require a fairness audit aligned with algorithmic-hiring literature [22, 23, 24].

The per-sub-score breakdown is the most useful diagnostic result. Keywords and structure converge strongly, while clarity diverges. This explains why the heuristic can be fast, cheap, and operationally useful without replacing the LLM component entirely.

A final limitation concerns trend awareness. The approved topic includes current job-market trends, but the implemented system reasons over a static resume and the current text of a job description. It does not model posting-frequency changes, regional demand, salary movement, or temporal skill adoption. Those signals would require a separate labour-market data layer and are left to future work.

Bibliography

1. Foundational AI for organisational decision-making

[1] M. Stone, E. Aravopoulou, Y. Ekinici, G. Evans, M. Hobbs, A. Labib, P. Laughlin, J. Machtynger, and L. Machtynger, “Artificial intelligence (AI) in strategic marketing decision-making: a research agenda,” *The Bottom Line*, vol. 33, no. 2, pp. 183–200, 2020. <https://doi.org/10.1108/BL-03-2020-0022>

[2] S. Gupta, S. Modgil, S. Bhattacharyya, *et al.*, “Artificial intelligence for decision support systems in the field of operations research: review and future scope of research,” *Annals of Operations Research*, vol. 308, pp. 215–274, 2022. <https://doi.org/10.1007/s10479-020-03856-6>

[3] A. M. Rahmani, B. Rezazadeh, M. Haghparast, W.-C. Chang, and S. G. Ting, “Applications of artificial intelligence in the economy, including applications in stock trading, market analysis, and risk management,” *IEEE Access*, vol. 11, pp. 80769–80793, 2023. <https://doi.org/10.1109/ACCESS.2023.3300036>

[4] Y. Kang, Z. Cai, C. W. Tan, Q. Huang, and H. Liu, “Natural language processing (NLP) in management research: A literature review,” *Journal of Management Analytics*, vol. 7, no. 2, pp. 139–172, 2020. <https://doi.org/10.1080/23270012.2020.1756939>

2. Foundational information retrieval

[5] G. Salton and M. J. McGill, *Introduction to modern information retrieval*. New York: McGraw-Hill, 1983.

[6] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009. <https://doi.org/10.1561/15000000019>

[7] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

3. Skill extraction and labour-market taxonomies (ESCO, O*NET)

[8] M. le Vrang, A. Papantoniou, E. Pauwels, P. Fannes, D. Vandensteen, and J. De Smedt, “ESCO: Boosting job matching in Europe with semantic interoperability,” *IEEE Computer*, vol. 47, no. 10, pp. 57–64, 2014. <https://doi.org/10.1109/MC.2014.283>

[9] D. C. Kavargyris, K. Georgiou, E. Papaioannou, K. Petrakis, N. Mittas, and L. Angelis, “ESCOX: A tool for skill and occupation extraction using LLMs from unstructured text,” *Software Impacts*, vol. 25, art. 100772, 2025. <https://doi.org/10.1016/j.simpa.2025.100772>

[10] A. Herandi, Y. Li, Z. Liu, X. Hu, and X. Cai, “Skill-LLM: Repurposing general-purpose LLMs for skill extraction,” *arXiv preprint arXiv:2410.12052*, 2024. <https://arxiv.org/abs/2410.12052>

[11] S. Saroglou, K. Diamantaras, F. Preta, M. Delianidi, A. Benisis, and C. J. Meyer, “Enhancing job matching: Occupation, skill and qualification linking with the ESCO and EQF taxonomies,” *arXiv preprint arXiv:2512.03195*, 2025. <https://arxiv.org/abs/2512.03195>

[12] C. Panzaru and A. Grama, “Towards explicit soft skills labelling in ESCO through semantic NLP analysis,” *Journal for Labour Market Research*, vol. 59, no. 1, art. 18, 2025. <https://doi.org/10.1186/s12651-025-00409-x>

[13] M. de Groot, J. Schutte, and D. Graus, “Job posting-enriched knowledge graph for skills-based matching,” *arXiv preprint arXiv:2109.02554*, 2021. <https://arxiv.org/abs/2109.02554>

4. Resume parsing and information extraction

[14] K. Yu, G. Guan, and M. Zhou, “Resume information extraction with cascaded hybrid model,” in *Proceedings of the 43rd Annual Meeting of the Association for Computational Linguistics (ACL '05)*, 2005, pp. 499–506. <https://aclanthology.org/P05-1062/>

5. AI-based career recommendation systems

[15] H. Al-Dossari, F. A. Nughaymish, Z. Al-Qahtani, M. Alkahlifah, and A. Alqahtani, “CareerRec: A machine learning approach to career path choice for information technology graduates,” *Engineering, Technology & Applied Science Research*, vol. 10, no. 6, pp. 6589–6596, 2020. <https://doi.org/10.48084/etasr.3821>

6. Large language models in recruitment

[16] C. Gan, Q. Zhang, and T. Mori, “Application of LLM agents in recruitment: A novel framework for resume screening,” *arXiv preprint arXiv:2401.08315*, 2024. <https://arxiv.org/abs/2401.08315>

[17] F. P.-W. Lo, J. Qiu, Z. Wang, H. Yu, Y. Chen, G. Zhang, and B. Lo, “AI hiring with LLMs: A context-aware and explainable multi-agent framework for resume screening,” in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2025, pp. 4184–4193. <https://doi.org/10.1109/CVPRW67362.2025.00402>

[18] R. V. K. Bevara, N. R. Mannuru, S. P. Karedla, B. D. Lund, T. Xiao, H. Pasem, S. C. Dronavalli, and S. Rupeshkumar, “Resume2Vec: Transforming applicant tracking systems with intelligent resume embeddings for precise candidate matching,” *Electronics*, vol. 14, no. 4, art. 794, 2025. <https://doi.org/10.3390/electronics14040794>

[19] A. Jagadeesh, C. R. Shankar, S. N. Patel, M. Levantesi, G. Semeraro, and E. W. De Luca, “AI-driven resume analysis and enhancement using semantic modeling and large language feedback loops,” in *Proceedings of CLiC-it 2025*, 2025. <https://aclanthology.org/2025.clicit-1.51/>

7. Semantic matching and sentence embeddings

[20] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of EMNLP-IJCNLP 2019*, Hong Kong, 2019. <https://arxiv.org/abs/1908.10084>

[21] V. Zadykian, B. Andrade, and H. Afli, “Towards explainable job title matching: Leveraging semantic textual relatedness and knowledge graphs,” *arXiv preprint arXiv:2509.09522*, 2025. <https://arxiv.org/abs/2509.09522>

8. Bias and fairness in algorithmic hiring

[22] A. Köchling and M. C. Wehner, “Discriminated by an algorithm: A systematic review of discrimination and fairness by algorithmic decision-making in the context of HR recruitment and HR development,” *Business Research*, vol. 13, no. 3, pp. 795–848, 2020. <https://doi.org/10.1007/s40685-020-00134-w>

[23] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, “Mitigating bias in algorithmic hiring: Evaluating claims and practices,” in *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (FAccT ’20)*, 2020, pp. 469–481. <https://doi.org/10.1145/3351095.3372828>

[24] Z. Chen, “Ethics and discrimination in artificial intelligence-enabled recruitment practices,” *Humanities and Social Sciences Communications*, vol. 10, art. 567, 2023. <https://doi.org/10.1057/s41599-023-02079-x>

9. Retrieval-augmented generation

[25] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2023. <https://arxiv.org/abs/2312.10997>

10. Web frameworks and system tooling

Documentation references for the engineering chapters; cited as official documentation footnotes rather than as numbered bibliography entries.

FastAPI documentation. <https://fastapi.tiangolo.com>

TanStack Start / Router documentation. <https://tanstack.com/router>

React 19 documentation. <https://react.dev>

Vite documentation. <https://vite.dev>

SQLAlchemy 2.0 documentation. <https://www.sqlalchemy.org>

Alembic migration documentation. <https://alembic.sqlalchemy.org>

PostgreSQL documentation. <https://www.postgresql.org/docs/>

Google Cloud Vertex AI Gemini documentation. <https://cloud.google.com/vertex-ai/generative-ai/docs>

Resend transactional email API. <https://resend.com/docs>

Appendices

Appendix A — Source listings

Excerpted listings of the most thesis-relevant code. Each listing is a representative excerpt; full source files of the same name in the repository contain additional helpers, error handling, and inline documentation that are not reproduced here in the interest of length. Each listing is preceded by its file path so the reader can navigate to the corresponding location in the public repository at the moment of submission.

A.1 Strong heuristic v2 — public entry point

`backend/app/services/quality_signals_v2.py`

```

def build_resume_prepass_v2(
    resume_text: str,
    job_description: str | None = None,
    target_role_label: str | None = None,
) -> ResumePrepassV2:
    text = resume_text or ""
    sections = _detect_sections(text)
    bullets = [m.group(1) for m in _BULLET_RE.finditer(text)]
    tokens = _tokenize(text)

    resume_skills = _normalise_skills(text)
    matched: list[str] = []
    missing: list[str] = []
    matched_locations: list[tuple[str, str | None]] = []
    fuzzy_count = 0

    # ... (full body in the source file; see Section 4.2 of the thesis for the
    # accompanying methodological description)

```

A.2 BM25 scoring

backend/app/services/quality_signals_v2.py

```

def _bm25_score(
    resume_tf: dict[str, int],
    query_terms: list[str],
    idf: dict[str, float],
    avg_dl: float = 350.0,
    k1: float = 1.5,
    b: float = 0.75,
) -> float:
    """Standard BM25 score of a query against a single document's term-frequencies."""
    if not query_terms:
        return 0.0
    dl = sum(resume_tf.values()) or 1
    score = 0.0
    for term in query_terms:
        tf = resume_tf.get(term, 0)
        if tf == 0:
            continue
        term_idf = idf.get(term, math.log((1 + 0.5) / (0 + 0.5) + 1))
        norm = tf * (k1 + 1) / (tf + k1 * (1 - b + b * dl / avg_dl))
        score += term_idf * norm

```

return score

A.3 Runtime mode toggle

```

backend/app/services/runtime_settings.py

ScoringMode = Literal["blended", "heuristic"]
_VALID_MODES: tuple[str, ...] = ("blended", "heuristic")

_initial_mode: ScoringMode = "blended"
configured = getattr(settings, "SCORING_MODE", "blended")
if configured in _VALID_MODES:
    _initial_mode = configured

_current_mode: ScoringMode = _initial_mode

def get_scoring_mode() -> ScoringMode:
    return _current_mode

def set_scoring_mode(mode: str) -> ScoringMode:
    global _current_mode
    if mode not in _VALID_MODES:
        raise ValueError(f"unknown scoring mode: {mode!r}")
    _current_mode = mode

return _current_mode

```

A.4 Shared tool pipeline

backend/app/services/tool_pipeline.py

```

async def run_tool_pipeline(
    *,
    tool_name: str,
    service_fn: Callable[..., Awaitable[dict[str, Any]]],
    service_kwargs: dict[str, Any],
    label_fn: Callable[[dict[str, Any]], str],
    resume_text: str,
    job_description: str | None = None,
    feedback: str | None = None,
    parent_run_id: str | None = None,
    workspace_id: str | None = None,
    linked_context_ids: list[str] | None = None,
    current_user: User | None = None,
    db: Session,
    cache_extra_keys: dict[str, str] | None = None,
) -> dict[str, Any]:
    """Shared pipeline: sanitize -> cache -> service -> fallback -> persist -> respond."""
    # ... (sanitisation, cache lookup, service call, persistence, observability)

```

Appendix B — Evaluation dataset description

The evaluation dataset used in Chapter 4.3 contains $N = 30$ resumes and $M \approx 20$ job descriptions, organised into six role tracks: backend engineering, frontend engineering, full-stack engineering, data analytics, product design, and product management.

B.1 Resume synthesis procedure

Each resume was synthesised from a templated structure that fills role-track-appropriate content into a fixed skeleton: a two-sentence summary, two or three employment entries with three responsibilities and one quantified achievement each, a skills list, a short Projects section with two open-source or side-project lines, and an education block. The synthesis script seeds the templated fields from a curated content library inside the script — role-track skill pools, action-verb-led outcome phrasings, fictional employer names, and a fixed candidate-name pool. Skill density and seniority increase monotonically across the five resumes per track so that the evaluation set spans junior, mid, and senior levels.

The synthesis script is `scripts/synthesise_eval_dataset.py`. It is fully deterministic: a fixed seed (`SEED = 42`) produces the same 30 resumes and 20 job descriptions on every run, which makes the evaluation manifest reproducible from the script alone.

B.2 Job description synthesis procedure

Job descriptions were also synthesised from templates rather than scraped from public job boards. Each description draws responsibilities from the same role-track pool used for resumes, and the requirement list is composed from a track-specific must-have skill set sized by seniority (four to six core skills plus one to three secondary skills, with years-of-experience banding by junior, mid, and senior). Synthesising rather than scraping preserves a clean comparison surface (the same skill vocabulary in resume and JD), avoids any privacy or licence concern, and is a deliberate choice that limits external generalisability — see Section 4.4 of Chapter 4 for the limitation framing.

B.3 Pair construction

The evaluation harness `scripts/eval_scoring.py` pairs each resume with every job description inside the same role track. The JD distribution is three JDs in four tracks and four JDs in two tracks, which produces an exact total of one hundred (resume, JD) pairs.

B.4 Pair manifest

The manifest produced by the synthesis script at `thesis/eval-dataset.json` is a list of one hundred objects with the shape `{id, resume, jd, role_track}`. The id is composed as `r-<track>-<NN>__jd-<track>-<NN>` so that every row is traceable back to its constituent resume and job description, and every per-pair result in Section 4.3 can be re-queried by id.

Appendix C — Screenshots of the running system

The screenshots in this appendix were captured at 1920×1080 viewport against the deployed system. The intent of the appendix is to demonstrate the engineering artefact — the running tools — rather than to document generic authentication UI; login forms, OAuth flows, and account-creation pages are therefore omitted in favour of screens that exhibit the analytical and generative tools described in Chapter 3 and the runtime scoring-mode switch documented in Section 4.2.9.

C.1 Landing page

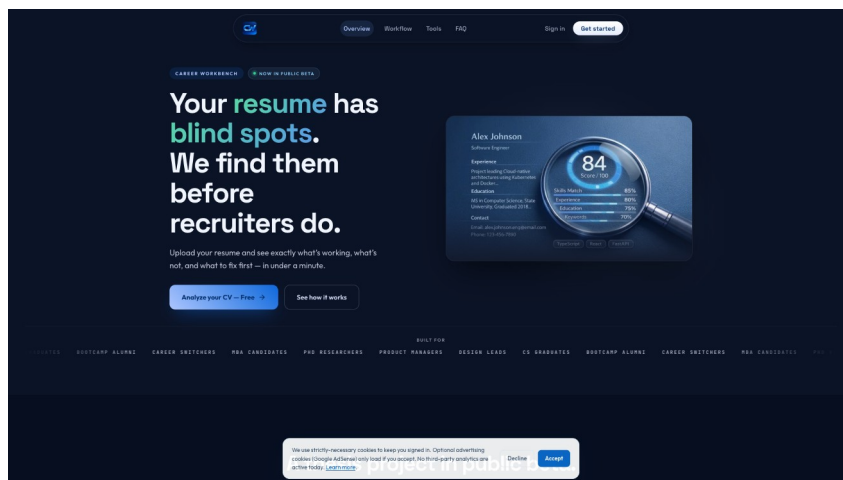


Figure C.1 — Career Workbench landing page (signed-out, production deployment).

Figure C.1: The public landing page at <https://thecareerworkbench.com>, presenting the six career-tooling pillars and the primary call-to-action to sign in or proceed as guest.

C.2 Resume Analyzer input page

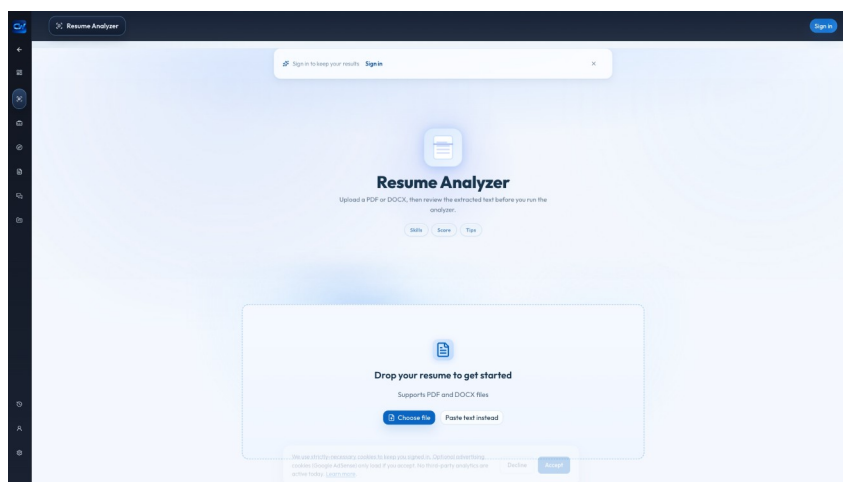


Figure C.2 — Resume Analyzer input page, guest mode.

Figure C.2: The Resume Analyzer input page (/resume) as it renders for a guest visitor. The input surface accepts a free-text or uploaded resume, with an optional target job description, before invoking the analytical pipeline described in Chapter 3.2.

C.3 Resume Analyzer result page (blended mode)

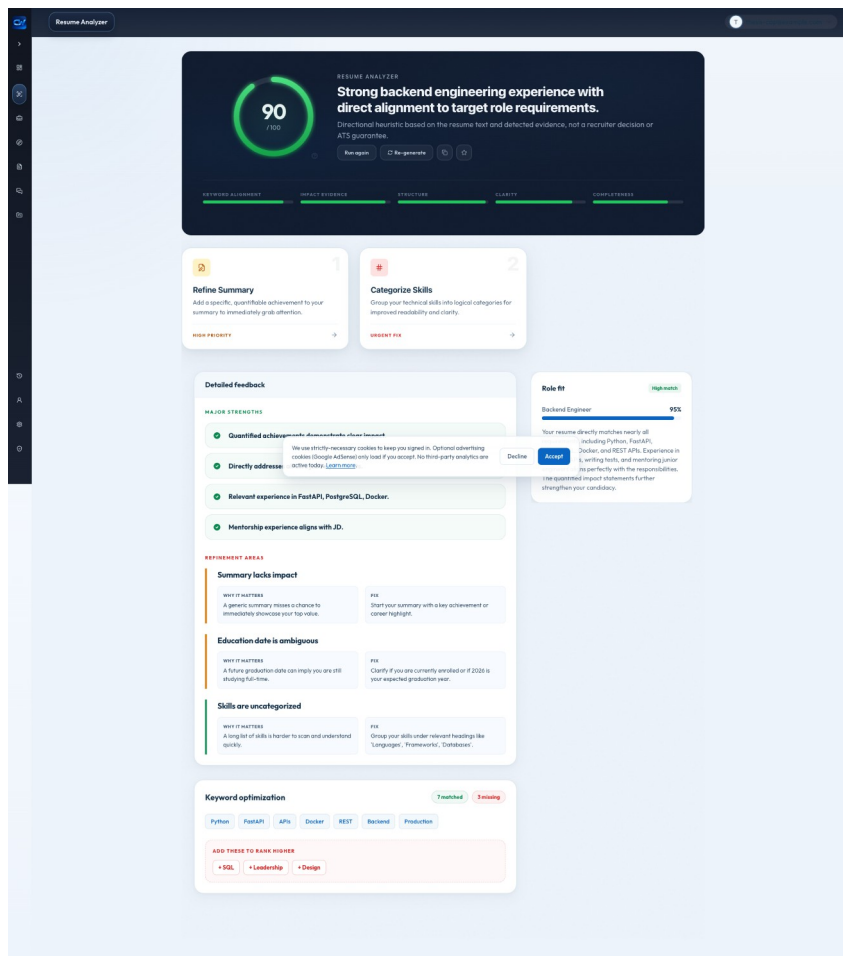


Figure C.3 — Resume Analyzer result page in blended mode.

Figure C.3: The Resume Analyzer result page after a blended-mode (heuristic 40 % + LLM 60 %) run on the synthetic backend-engineer resume described in Chapter 4 paired with a representative backend-engineer job description. The page renders the overall score, the five-axis sub-score breakdown, the deterministic issues list, the prioritised top actions, and the role-fit narrative — the response shape documented in Chapter 3.2.3.

C.4 Job Match result page

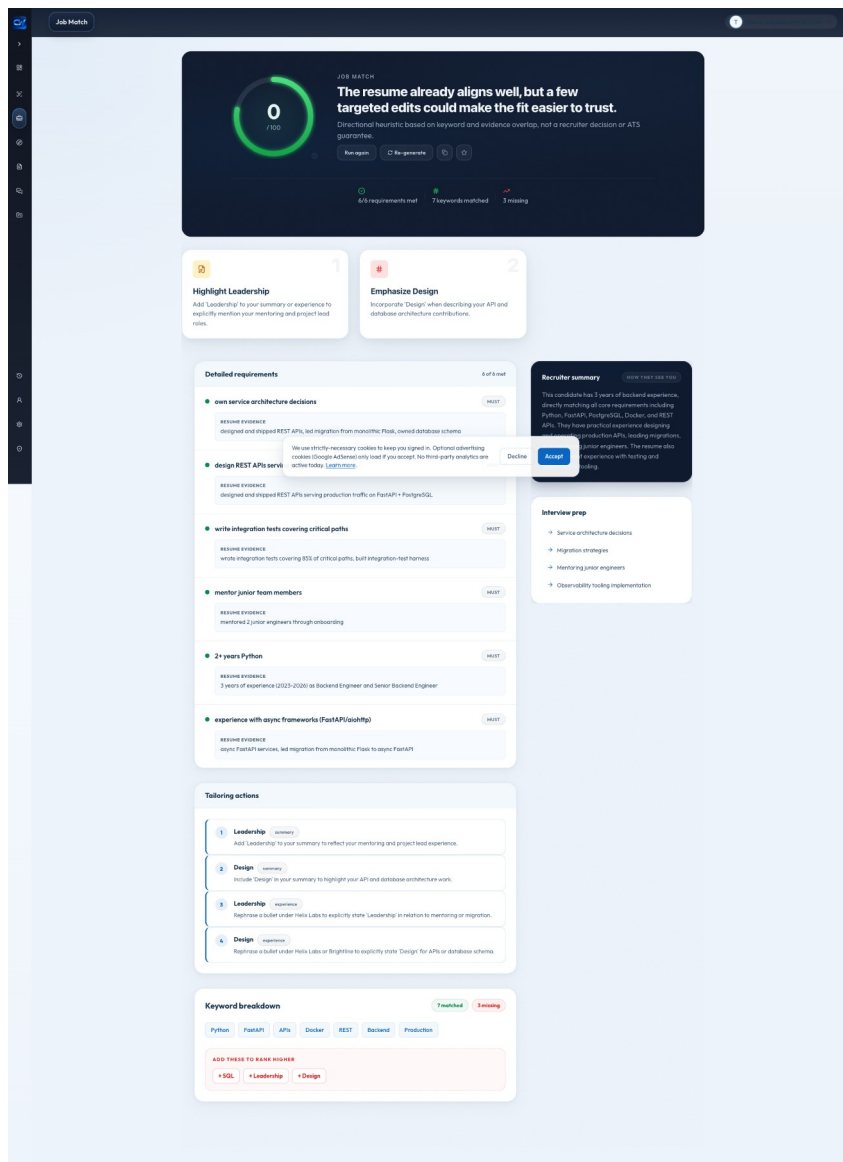


Figure C.4 — Job Match result page on the same resume / job-description pair.



Figure C.4: The Job Match result page on the same input pair as Figure C.3. The view exposes the matched-keyword set, the missing-keyword set, the deterministic verdict, the recruiter-style summary, and the requirement-by-requirement breakdown produced by the prompt described in Chapter 3.3.3.

Appendix D — Configuration reference

D.1 Backend environment variables

The following environment variables are read at process start and configure the deployed system. Defaults shown in parentheses; consult `backend/app/config.py` for the authoritative list.

Variable

Purpose

Default

DATABASE_URL

PostgreSQL DSN

sqlite:///career_platform.db

SECRET_KEY

JWT signing secret

(must override)

ACCESS_TOKEN_EXPIRE_MINUTES

Access-token lifetime

30

REFRESH_TOKEN_EXPIRE_DAYS

Refresh-token lifetime

7

LLM_PROVIDER

LLM dispatch backend

vertex

LLM_MODEL

Primary Gemini model

gemini-2.5-flash

LLM_PRACTICE_MODEL

Cheaper model used for interview practice feedback

(empty → falls back to LLM_MODEL)

VERTEX_PROJECT_ID

Google Cloud project

(must override)

VERTEX_LOCATION

Vertex region

us-central1

BLENDED_SCORING_ENABLED

Apply 60/40 LLM/heuristic blend

true

SCORING_MODE

Scoring core mode (blended or heuristic) — added with the comparative study

blended

RESULT_CACHE_ENABLED

In-memory cache

true

RESULT_CACHE_TTL_SECONDS

Cache TTL

3600

CORS_ORIGINS

Allowed CORS origins

localhost defaults

FRONTEND_URL

Canonical frontend URL

(set in production)

GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET / GOOGLE_REDIRECT_URI

Google OAuth

(must override)

RESEND_API_KEY

Transactional email

(must override)

SENTRY_DSN

Error tracking

(empty disables)

D.2 Administrative endpoints

The administrative HTTP API used for the comparative study is exposed under `/api/v1/admin/`:

GET `/api/v1/admin/scoring-mode` → `AdminScoringModeResponse`

POST `/api/v1/admin/scoring-mode` with body `{ "mode": "blended" | "heuristic" }` → returns the updated `AdminScoringModeResponse`

The response shape is:

```
{
    "mode": "blended",
    "heuristic_version": "v1",
    "blended_weight_heuristic": 0.4,
    "blended_weight_llm": 0.6
}
```

Both endpoints require an authenticated administrator (the `users.is_admin` flag controls access) and are rate-limited at 60 requests per minute per administrator. The runtime mode change is held in process memory only; restarting the backend restores the configured default from the `SCORING_MODE` environment variable.

D.3 Running the evaluation harness

```
$ cd backend
$ SCORING_MODE=blended RESULT_CACHE_ENABLED=false \
  python ../scripts/eval_scoring.py
$ python ../scripts/analyze_eval_results.py > ../thesis/chapter-04-results.md
```

The harness writes `thesis/eval-results.json`; the analyzer prints the Markdown tables ready to paste into Chapter 4.3.